

Biometric Age Prediction — Languages & Methods Guide

System: Real-Time Computer Vision & Explainable AI (XAI) | Author: Frontend Engineering Team

1. Core Languages & Web Technologies (Top-to-Bottom)

The application is engineered without third-party framework overhead, leveraging standard web standards for maximum execution speed, zero installation requirements, and 60 FPS hardware acceleration:

Technology	Standard / Version	Key Features Utilized	Role in Project
HTML5	W3C HTML5 Recommendation	Semantic tags, <video>, <canvas>, <input type='file'>, ARIA dialogs	Provides document structure, multimedia stream bindings, and accessible modal overlays.
Vanilla CSS3	CSS Level 3 / Custom Properties	CSS Variables, CSS Grid, Flexbox, Keyframes, transform: scaleX(-1)	Controls the Royal Blue & White theme, hardware-accelerated animations, and responsive layouts.
Vanilla JavaScript	ECMAScript 2022 (ES6+)	Promises, async/await, arrow functions, modules, destructuring	Implements client-side application logic, vision processing loop, and UI event controllers.
Web MediaDevices API	W3C Media Capture Streams	navigator.mediaDevices.getUserMedia()	Captures hardware webcam video feed with user constraints (1280x720 ideal resolution).
HTML5 Canvas 2D API	W3C 2D Context API	CanvasRenderingContext2D paths, strokes, transforms, text	Draws real-time polygon wireframe, cyber brackets, and floating age badge below face at 60 FPS.
Google MediaPipe	FaceMesh v0.4 (Wasm/WebGL)	468 3D facial landmark regression model	Client-side neural network tracking facial coordinates directly inside the browser.
HTML5 File API	W3C File API	FileReader.readAsDataURL()	Decodes local image uploads into base64 data for photo-mode age prediction.
Web Storage API	W3C Web Storage	window.localStorage	Persists Light / Dark theme selection across browser sessions.

2. Mathematical, Algorithmic & Computer Vision Methods

A. Exponential Moving Average (EMA) Smoothing Algorithm:

Raw webcam landmark coordinates often suffer from high-frequency hardware noise (jitter). The system applies an Exponential Moving Average filter on each frame: $P_smoothed(t) = \alpha \cdot P_raw(t) + (1 - \alpha) \cdot P_smoothed(t-1)$ (where $\alpha = 0.45$), guaranteeing buttery-smooth box and landmark movement at 60 FPS.

B. Geometric Landmark Normalization & Mirror Compensation:

Because front-facing cameras are naturally mirrored via CSS (transform: scaleX(-1)), the JavaScript coordinates undergo horizontal inversion mapping: $X_canvas = (1.0 - X_norm) \times Width_canvas$ and $Y_canvas = Y_norm \times Height_canvas$.

C. Delaunay-Style Facial Triangulation Topology:

The 31 key anatomical nodes are mapped into polygon facets (forehead hexagon, orbital triangles, nasal bridge, nasolabial lines, and chin contour) strictly **inside the facial boundary**, reflecting standard craniofacial anthropometry.

D. Explainable AI (XAI) Biological Reasoning:

Instead of returning an opaque prediction, the system segments the face into 4 biological aging zones: (1) Periocular/Canthal lines, (2) Nasolabial sulcus prominence, (3) Forehead skin texture smoothness, and (4) Mandibular contour firmness.

3. Exhaustive Codebase Methods & Functions (Top-to-Bottom)

Function Name	Parameters	Description & Execution Workflow
initTheme()	None	Reads saved theme preference from localStorage (defaults to 'light') and sets up click listener on toggle button.
applyTheme(t)	t (string: 'light' 'dark')	Applies data-theme attribute on <html> element and updates theme text and sun/moon icon.
initFaceMesh()	None	Instantiates Google MediaPipe FaceMesh instance, configures tracking confidence (0.5), and registers callback.

onFaceMeshResults(res)	results (Object)	Primary vision callback. Extracts 468 landmark points, calculates bounding box extremes, applies EMA smoothing, triggers canvas rendering, and updates UI status.
drawRealFaceMesh()	box, pts, age, conf	Renders yellow polygon wireframe lines, square landmark nodes, cyan corner bounding box, and the floating age badge 12px strictly below the face.
drawRoundedRect()	ctx, x, y, w, h, r	Mathematical helper using quadratic Bezier curves (quadraticCurveTo) to render smooth rounded HUD badges.
startCamera()	None (async)	Requests camera permissions via getUserMedia, binds stream to <video>, synchronizes canvas, and starts the processing loop.
stopCamera()	None	Iterates over active media stream tracks and calls track.stop(), clears canvas, and restores standby screen.
toggleFreeze()	None	Pauses/resumes the real-time frame processing loop for visual inspection and grading.
handleImageUpload(f)	file (File Object)	Uses FileReader to read local user photo, halts live camera, and switches application into 'Photo Mode'.
renderUploadedPhoto()	img (Image Object)	Calculates aspect-ratio fit, centers image on canvas, detects face boundaries, and overlays biometric mesh and age tag.
updateExplainabilityReason()	age, customReason	Generates dynamic textual rationale explaining why the model predicted that specific age based on the 4 biological facial markers.
openXaiModal() / closeXaiModal()	None	Controls visibility and popup animations of the Explainable AI (XAI) modal dialog.
startContinuousProcessing()	None (async)	60 FPS non-blocking animation loop using requestAnimationFrame that passes video frames to MediaPipe.
runFallbackDetection()	timestamp (DOMHighRes)	Offline fallback simulator that ensures face tracking runs seamlessly even in environments without internet access.

4. Architectural Justification & Interview Q&A;

Q: Why Vanilla HTML/CSS/JS instead of React or Vue?

Answer: Vanilla JS runs natively in any browser with zero compilation or installation steps. React introduces virtual DOM overhead that adds unnecessary latency when updating 60 FPS canvas coordinates. Vanilla JS provides direct, raw access to the GPU-accelerated Canvas 2D API.

Q: Why client-side MediaPipe instead of sending video to Python OpenCV backend?

Answer: Sending 60 frames per second over HTTP/WebSocket creates massive network bandwidth bottlenecks and latency. Executing face detection on the client via WebAssembly/WebGL gives instant 60 FPS feedback with 0ms network lag.

Q: How is the backend integrated?

Answer: The architecture is decoupled. When the Python PyTorch/FastAPI backend is ready, the frontend simply makes a single POST /api/predict-age request with the captured facial frame, and updates the age badge and explanation modal dynamically.